



Software Quality Engineering –SESQ-345



Software Quality Factors and Product Quality Models

DR. NARGIS FATIMA

ASSISTANT PROFESSOR

DEPARTMENT OF SOFTWARE ENGINEERING , FEC

Agenda

Software Quality Factors

Software Quality models

- MacCall's
- Boehm's
- ISO/IEC 25010:2011 {the International Electrotechnical Commission}
- Etc.



Software Quality Factors

The various factors, which influence the software, are termed as software factors, Software quality attributes, or characteristics

Software Quality Factors

- Groups of software quality factors have been presented in several **software quality models**.
- Several subfactors for each of these factors have been suggested.
- These subfactors are expected to be measurable.

SOFTWARE QUALITY MODELS

McCall's classic model for software quality factors-1977

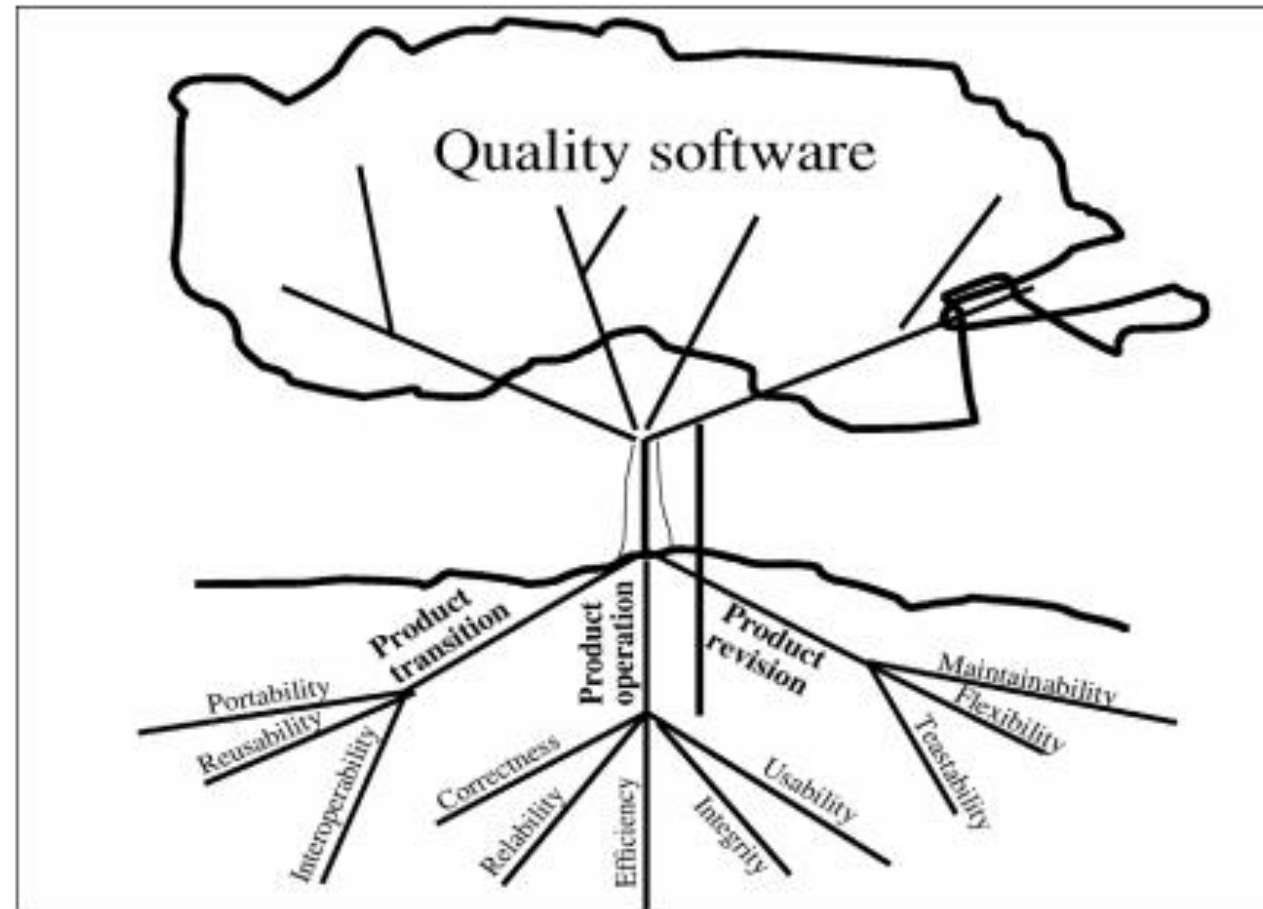


McCall's classic model for software quality factors-1977

McCall's factor model classifies all software requirements into

- 11 software quality factors.
- 11 factors are grouped into three categories:
 - product operation
 - product revision
 - product transition

McCall's software quality factor model tree



McCall's classic model for software quality factors-1977

Product operation

- five software quality factors are included in the product operation category, all of which deal with requirements that directly affect the daily operation of the software
 - Correctness
 - Reliability
 - Efficiency
 - Integrity
 - Usability

McCall's classic model for software quality factors-1977

Product operation -Correctness

- Correctness requirements are related to the **outputs of software systems**, such as a query display of a customer's balance in the sales accounting information system.

McCall's classic model for software quality factors-1977

- Product operation- Correctness
 - This output specification is usually multidimensional; some common dimensions are:
 - The required accuracy of the output.
 - This may be adversely affected by inaccurate data or inaccurate calculations. For example, the probability of non-accurate inventory information, containing one or more mistakes, will not exceed 1%
 - The required completeness of the output information
 - This may be adversely affected by incomplete data. For example, the probability of missing data from a club member's record: the number of his cellular phone or home address will not exceed 1%

McCall's classic model for software quality factors-1977

Product operation- Correctness

- The required response time, defined as the time needed to obtain the requested information, or as the requested reaction time of the software
- Example: Response time for queries regarding the inventory item will be on average less than 2 seconds

McCall's classic model for software quality factors-1977



Product operation-Reliability

- Reliability requirements deal with failures to provide service.
- They determine the maximum allowed software system failure rate, the maximum allowed percentage of a software system's downtime, and the maximum allowed recovery times.
- The requirements can refer to the entire system or to one or more of its separate functions
- Examples. The failure frequency of a heart-monitoring unit operating in a hospital's intensive care ward is required to be less than one in 20 years. Its heart attack detection function is required to have a failure rate of less than one per 100 years.



McCall's classic model for software quality factors-1977

Product operation-Efficiency

- Efficiency
- Describe the extent to which the software makes optimal use of resources, the speed with which the system executes, and the memory it consumes for its operation.

McCall's classic model for software quality factors-1977

Product operation

- Integrity
 - Integrity requirements deal with the software system security, that is, requirements to prevent unauthorized access, to distinguish between the majority of personnel only allowed to see the information (read only permit), and a limited group who will be allowed to add and change data (write permit)
- Aim is to avoid inconsistency of data

McCall's classic model for software quality factors-1977

Product operation-Usability

- Usability requirements deal with the scope of staff resources needed to train a new employee and to operate the software system.
- The usability requirements relate to the
 - (a) operation usability, the productivity of the user, that is, the average number of transactions performed per hour
 - (b) training usability, the average time spent training a new employee.

McCall's classic model for software quality factors-1977

Product operation

- Usability
- Example
 - The software usability requirements for the new help desk system initiated by a home appliance manufacturing company lists the following specification requirements:
 - (1) A staff member should be able to handle at least 60 service calls a day.
 - (2) Training a new employee will take no more than 2 days (16 training hours), at the end of which the trainee will be able to handle 45 service calls a day

McCall's classic model for software quality factors-1977

Product revision

- Three quality factors comprise the product revision category. These factors deal with those requirements that affect the complete range of software maintenance activities:
 - Maintainability
 - Flexibility
 - Testability

McCall's classic model for software quality factors-1977

Product revision-Maintainability

- Maintainability requirements determine the efforts needed by users and maintenance personnel to identify the reasons of a software failure, to correct the failure, and to verify the success of the correction.
- This factor's requirements refer to the modular structure of software, the internal program documentation, and the programmer's manual, among other items.

McCall's classic model for software quality factors-1977

Product revision-Maintainability

- Example Typical maintainability requirements:
 - The size of a software module will not exceed 30 statements.

McCall's classic model for software quality factors-1977

Product revision-Flexibility

- The flexibility factor deals with the capabilities and efforts required to support adaptive maintenance activities.
- Example: The software specifications included the following flexibility requirements:
 - the software should be suitable for teachers of all subjects and school levels (elementary, middle, and high school).
 - Non-professionals should be able to use system to create new types of reports according to the schoolmaster's requirements

McCall's classic model for software quality factors-1977

Product revision-Testability

- Testability requirements deal with the testing process of a software system, as well as with its operation.
- Example
 - An industrial computerized control unit is programmed to calculate various measures of the production status, report the performance level of the machinery, and operate a warning signal in predefined situations.
 - One testability requirement demanded was to develop a set of standard test data, including the expected correct reactions of the system in each case. This standard test data is to be run every morning before production begins, to verify that the computerized unit responds properly

McCall's classic model for software quality factors-1977

Product transition

- three quality factors are included in the product transition category, a category that pertains to the adaptation of software to other environments, and its interaction with other software systems
 - Portability
 - Reusability
 - Interoperability

McCall's classic model for software quality factors-1977

Product transition

- Portability requirements relate to the adaptation of a software system to other environments consisting of different hardware, different operating systems.
- Example
 - A software package designed and programmed to operate in a Windows 10 environment is required to allow a low cost transfer to Linux environments.

McCall's classic model for software quality factors-1977

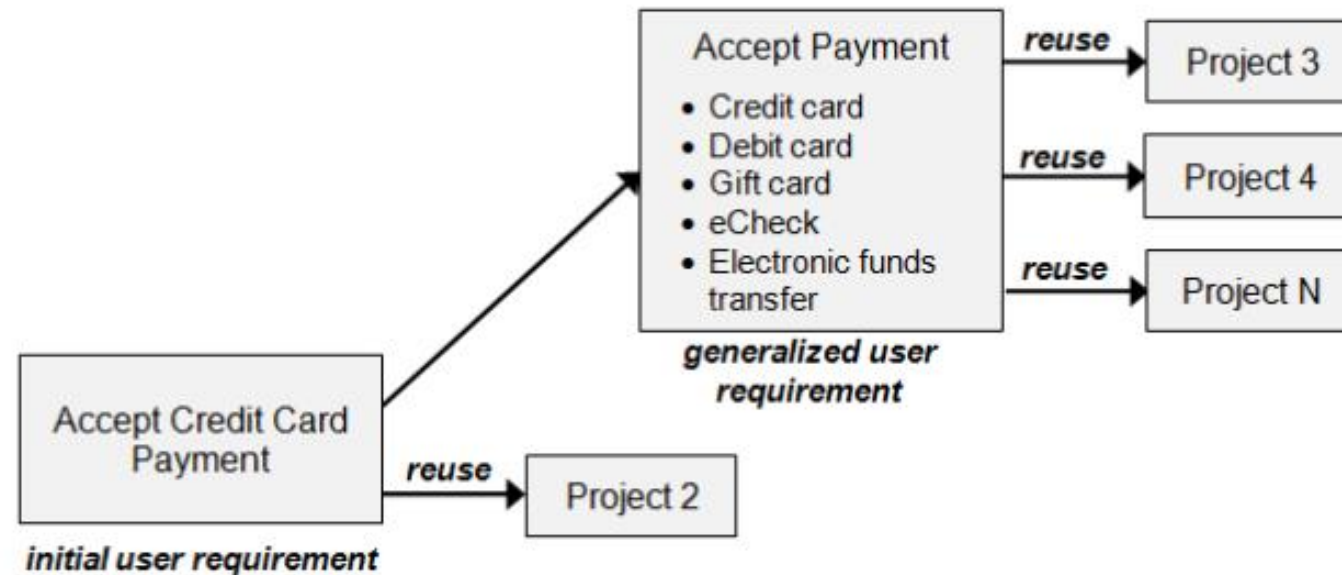
Product transition

- Reusability
 - is the use of a software module, or an entire application, taken from an existing software product in a new software project currently being developed.
 - The existing software module or entire application may be software former developed by the organization, or open source software, etc.

McCall's classic model for software quality factors-1977

Product transition

- Reusability
- Example



McCall's classic model for software quality factors-1977

Product transition-Interoperability

- focus on creating interfaces with other software systems or equipment firmware
- Example:
 - the laboratory information system is required to interface with a medical clinic information system, in order to transmit patients' test results automatically to the physician's clinic.

McCall's classic model for software quality factors-1977

Product transition-Interoperability

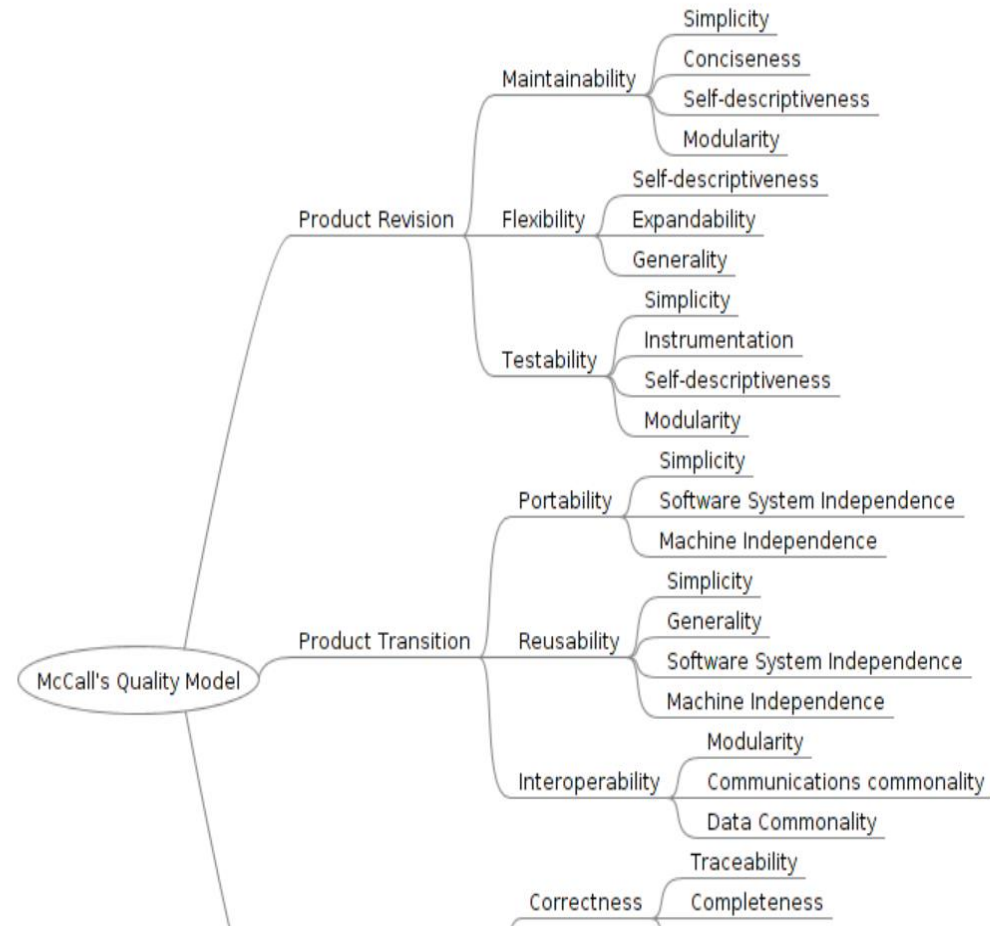
- Example, the firmware (embedded into the memory of hardware devices) of the production machinery and testing equipment interfaces with the production control software).
- Interoperability requirements sometimes specify the name(s) of the software or firmware to which an interface is required.
- Hospitals often use various medical devices such as heart monitors, infusion pumps, and ventilators, each with its own software interface. The software should be interoperable with these devices, allowing for the exchange of data in real-time. For example, the software should be able to receive vital signs data from a patient monitor and incorporate it into the patient's electronic record automatically.

McCall's classic model for software quality factors-1977

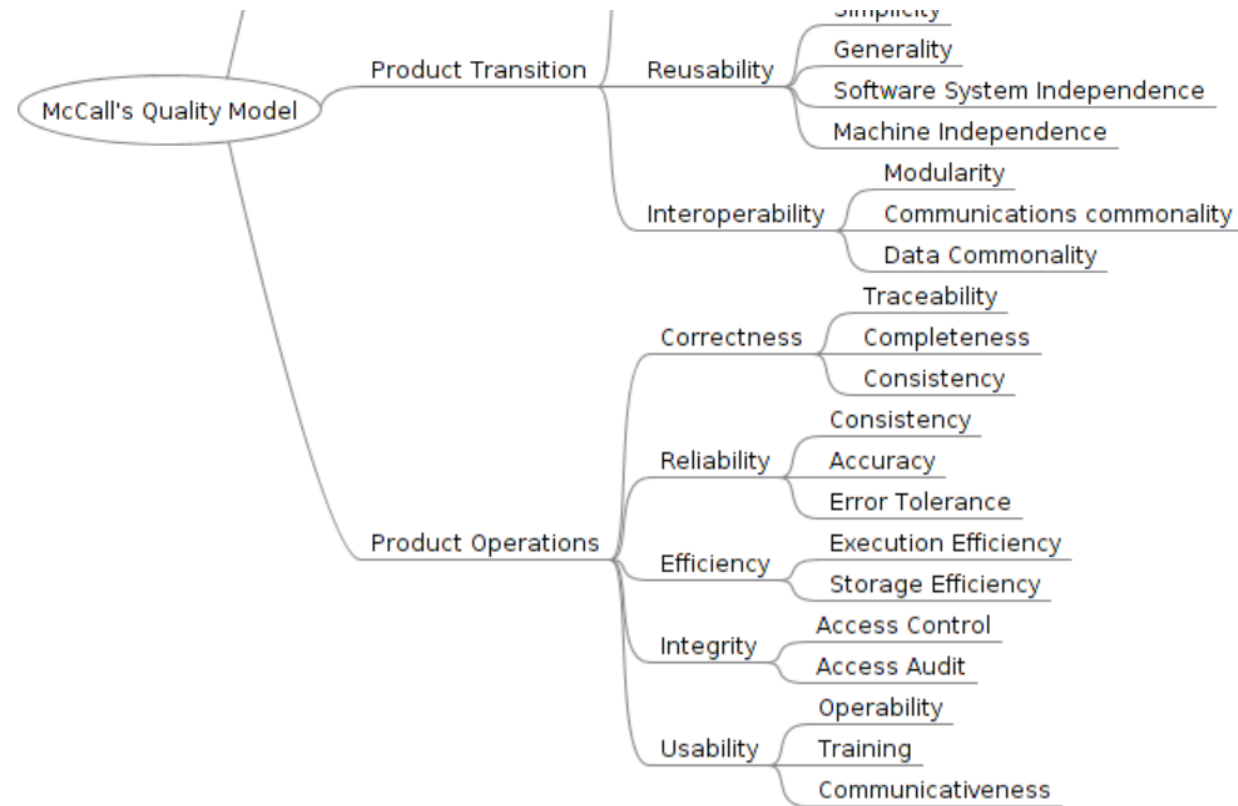


McCall's software quality factors model	
Product operation factors:	Correctness Reliability Efficiency Integrity Usability
Product revision factors:	Maintainability Flexibility Testability
Product transition factors:	Portability Reusability Interoperability

McCall's model cont.



McCall's model cont.



Variations of McCall's Models

Variations of McCall's Models

Several software quality factor models, alternatives to the classic McCall's model, have been developed during the last four decades.

- Boehm's model (Boehm et al., 1978),
- FURPS model (first proposed by Grady and Caswell, 1987),
- ISO/ IEC 25010:2011 (ISO/IEC, 2011)

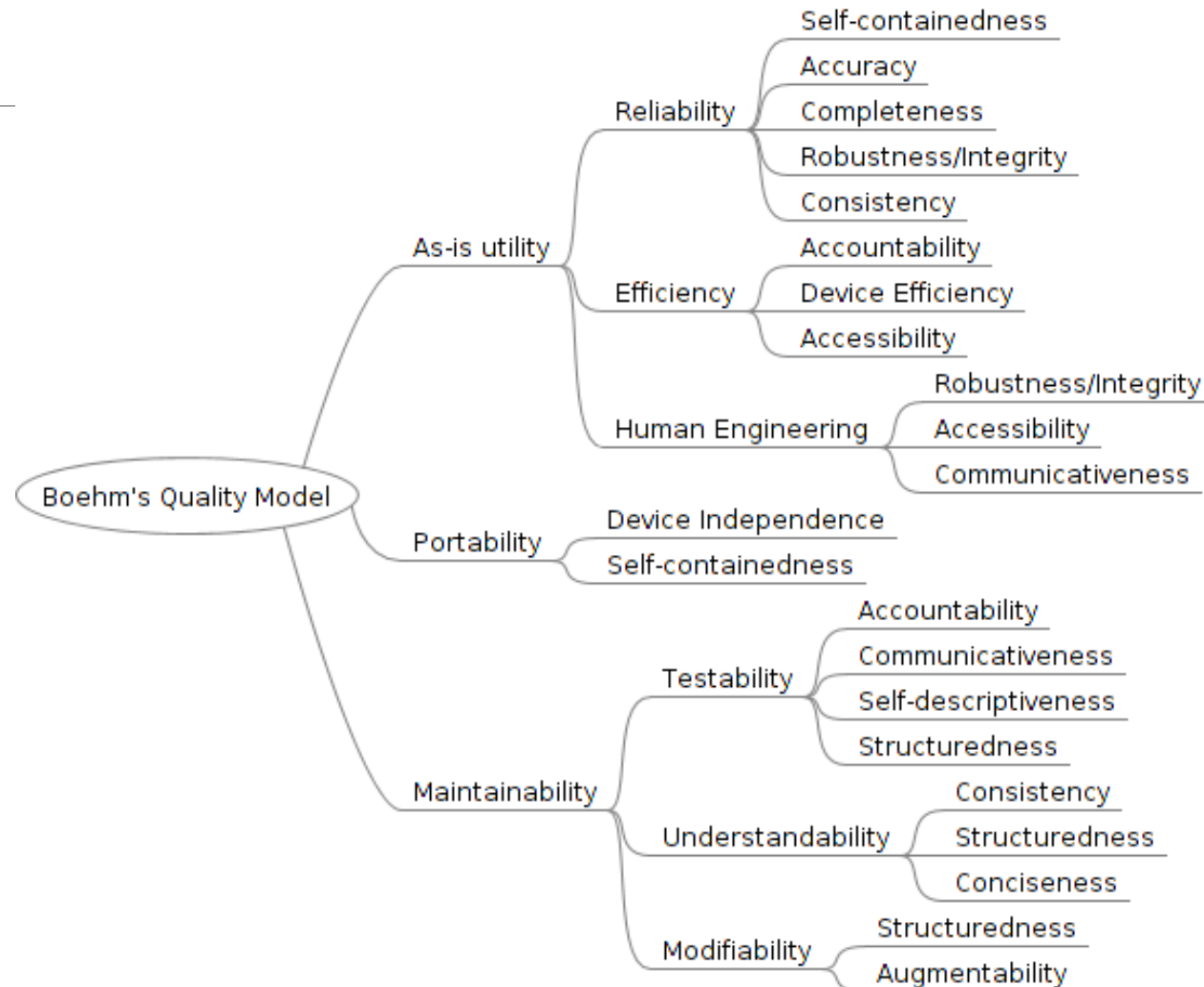
Boehm Model-1978

Boehm Model-1978

Boehm added new factors to McCall's model and emphasis on the maintainability of software product.

Boehm model represents the characteristics of the software product hierarchically in order to get contribute in the total quality.

Boehm Model Cont.

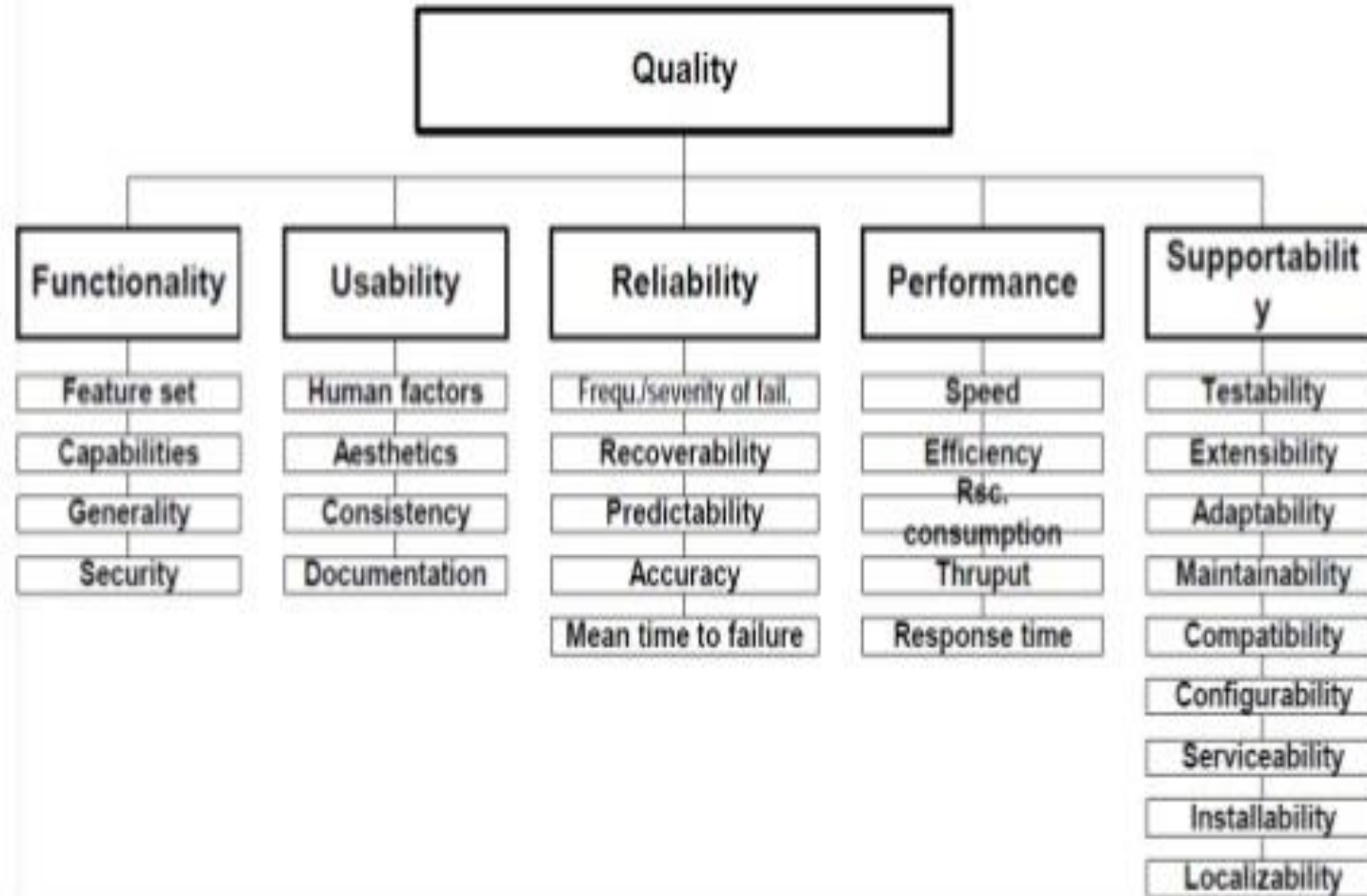


FURPS Model-1987

FURPS Model

- FURPS model was proposed by Robert Grady and Hewlett-Packard Co.
- The characteristics were classified into two categories according to the user's requirements,
 - Functional Requirements
 - Non-functional requirements.
 - Usability, reliability, performance, supportability.

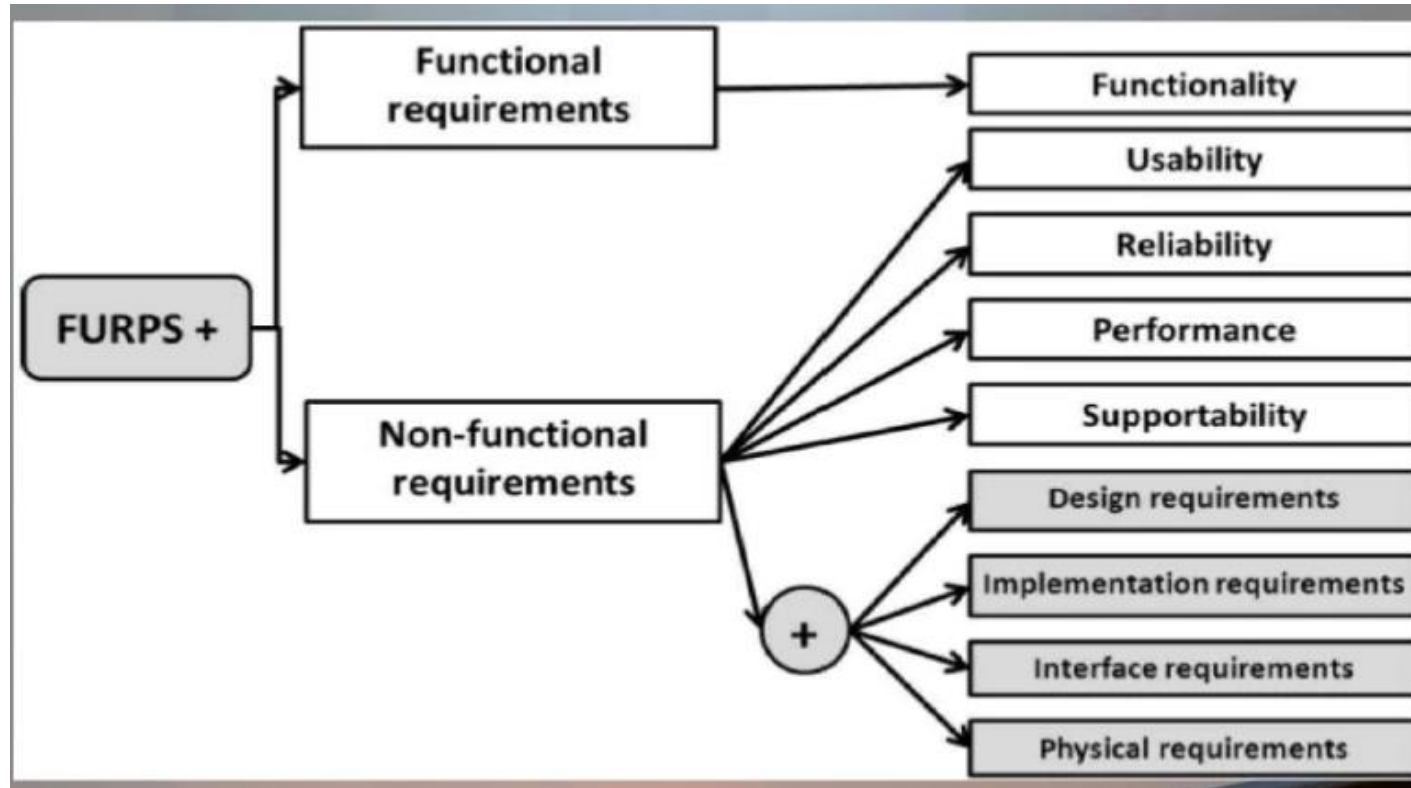
FURPS-1987



FURPS Model

- The model was extended by IBM Rational Software – into FURPS+.

FURPS-1992



ISO/IEC 25010:2011

ISO/IEC 25010:2011

The ISO/IEC 25010:2011 standard replaced the ISO/IEC IS 9126-1:2001 (ISO/ IEC, 2001)

Composed of eight factors, and shows a substantial similarity to McCall's model, as four out of its eight factors are included in McCall's model

Included Quality Factors

- Functional suitability
- Performance efficiency
- Compatibility
- Usability
- Reliability
- Security
- Maintainability
- Portability

ISO/IEC 25010:2011

Quality Factors

Functional suitability

- Functional suitability is the capability to fulfill the functions needed by the customer, stated or implied (not necessarily the specified requirement). It covers a wide variety of aspects of software use, including the accurate and complete production of needed results. A significant similarity exists between the functionality factor and the correctness, integrity and interoperability factors of McCall's model.

ISO/IEC 25010:2011

Performance efficiency

Software performance efficiency relates to the amount of hardware resources required to fulfill the software system tasks. The lower the amount of hardware resources, the higher the performance. A significant similarity exists between the performance factor and the efficiency factor of McCall's model.

ISO/IEC 25010:2011

Compatibility

Compatibility refers to the capability of a software system or component to (a) exchange information with other software systems or components and (b) perform other system



ISO/IEC 25010:2011

Security

It relates to the capability of a system product to protect the software system, data stores, and information produced from the reading, modification, or destruction by unauthorized persons or systems.

To accomplish this, the system controls access to it through levels of authorization to persons and software products, and denies unauthorized ones.

A significant similarity exists between the security factor and the integrity factor described in McCall's model